

AI Usage Declaration

Project: Project Mayhem Management **Course:** Introduction to Computer Science — HIT, Spring 2026 **Author:** (your name)

This document explains, honestly and in full, how AI tools were used throughout the development of this project — from idea to submission. It is written to match the course's requirement to declare *all* ways AI was used and which tools.

1. Tools used

Tool	Role in this project
Claude (Anthropic) — Opus 4.x	Primary assistant: design discussion, code drafting, debugging, learning, documentation
Claude Code (CLI / agent)	Ran in the project repo: read the source, edited files, ran the MSVC build, ran tests
Claude (Sonnet)	Used as a delegated sub-agent for a few well-specified, lower-risk implementation tasks, reviewed by me
Graphviz (<code>dot</code>)	Not AI — used to render the dependency diagrams the assistant generated as <code>.dot</code> source

No other AI services (e.g. ChatGPT, Copilot, Gemini) were used.

2. How AI was used, by phase

Ideation

I used the assistant to research how real project-management tools (Jira, Asana, Monday, MS Project, GanttProject) work and where users criticize them. That comparison is what produced the project's original angle: a **minimal terminal app with a genuine scheduling core** — a real Critical Path Method Gantt and a cross-project resource calendar — rather than a feature-heavy clone.

Planning & design

The architecture was worked out as a **dialogue**. The most important design decisions were debated with the assistant before any code was written, for example:

- **Who owns team members** — the company owns members; projects hold member *ids*, not member objects.

- **Per-project task ids** — task ids are unique only within a project (resolved via a lookup), not globally unique.
- **Task splitting model** — a lightweight "segment" inside a task vs. a recursive Task ("composite"). I asked "*why not make a subtask just a Task?*" and we reasoned through it together; I chose the lightweight leaf to keep task identity stable. The decision and its rejected alternative are documented.

The assistant proposed options and tradeoffs; **I made the calls.**

Implementation

Most modules were **drafted by the assistant from a precise specification that I wrote** (function signature, algorithm, constraints: C99, `/W4`, ASCII-only). I then read every function, asked it to explain anything I was unsure of, and required the build to stay at **zero warnings**. Several features were built in an extended session where the assistant implemented a roadmap step-by-step (cross-project member calendar, assignment policies, vacations, portfolio Gantt, role/privilege system, task splitting, an `id→pointer` performance index) — each step **unit-tested before moving on**.

Heavy mechanical work

AI was especially valuable for the **laborious, repetitive work** — the kind that is conceptually simple but tedious and error-prone to do by hand. Once I decided the approach, the assistant carried out the grind:

- **Pattern restructuring** — collapsing the five hand-written, near-duplicate menu loops into a single **generic** `run_table_menu` / `run_checklist` engine in `tui_framework` (driven by a `void *ctx` + function pointers). Mechanical, wide-reaching edits across every menu; I specified the target shape, the assistant applied it consistently and kept the build green.
- **Parsers and serializers** — the **YAML-ish save/load** for the directory bundle (`persistence.c`: field-by-field `fprintf` writers and `fgets` / `strtok` readers) and the **HTML report builder** (`html_builder` / `report_exporter`: document chrome, tables, progress bars, and the character escaping). This is exactly the boilerplate-heavy, easy-to-get-subtly-wrong code where an assistant saves hours — I reviewed the format and the edge cases.

I chose the design; the assistant did the heavy lifting of applying it everywhere.

Testing

The assistant wrote small, focused **unit-test harnesses** (one per feature: the CPM schedule, the cross-project calendar, the overlap resolver, the role privileges, etc.). I used these to confirm each change and to catch regressions in the others. The tests are throwaway harnesses, kept out of the submitted solution.

Debugging

When something was wrong I brought the symptom (a crash, a wrong number on the Gantt, a build error) and we narrowed the cause together — usually with the assistant asking me diagnostic questions rather than just handing me a fix. Real bugs found this way include a NULL-dereference in cycle detection on a forward-referenced task, and a pointer-truncation caused by a **missing** **#include** in a test harness.

Code review (including AI reviewing AI)

The assistant acted as a **second pair of eyes** on code I had written or accepted — flagging correctness bugs, ownership/ **free** mistakes, dead code, and places that could be simplified or de-duplicated. I treated its findings as suggestions to judge, not orders: some I applied, some I rejected after thinking them through. Several of the project's "smells / encapsulation / dead-code" cleanups came out of these review passes.

This also ran **AI-on-AI**: code generated by one agent was reviewed by a **separate, independent agent** before it reached me. In the larger build the work was split between a lead model and delegated sub-agents, with one model **adversarially checking another's output** — catching bugs and weak spots so that AI-written code was already scrutinised once by AI before I reviewed it myself. The competing-agents pass is a filter *before* my review, never a replacement for it.

Build validation

Because the assistant ran inside the project (Claude Code), it could **build the solution with MSVC and read the result** — compiling at **/W4**, surfacing warnings and link errors, and confirming a clean **0 warnings / 0 errors** after each change. That made the build the constant gate: nothing was considered done until it compiled clean and the test harnesses passed.

Guided / exploratory writing

On the parts I found **interesting or wanted to explore**, the mode shifted from "spec then generate" to **pair-writing** — I sketched a direction and we built it together, trying alternatives (e.g. the task-splitting model, the resource-overlap fixpoint, the cross-project calendar). Here AI was a thinking partner for exploration, not just a code generator: the back-and-forth is where several of the design decisions actually crystallised.

Comparing alternatives

A recurring use was asking the assistant to **lay out competing options with their trade-offs** so I could choose with the full picture in front of me — weighing them, not being handed a verdict. Examples that shaped the design:

- **Task splitting**: recursive **Task** ("composite") vs. a lightweight **segment** leaf — compared for identity stability and field duplication; I chose the leaf.

- **id → pointer lookup**: a direct index table vs. keeping the array sorted + **bsearch** — compared on rebuild cost and read-heavy access; I chose the index.
- **Unassignable-task cascade**: a callback during the loop vs. collect-then- resolve afterwards — compared on where interactivity belongs; I chose the latter.

The assistant supplied the comparison; the decision stayed mine.

Knowledge expansion

When I hit something I genuinely **did not know how to do**, I asked the assistant for **guidance** — to explain the technique, show the idiom, or point me at the right approach — and then implemented it myself. This is how I picked up unfamiliar C and Windows-specific patterns (e.g. `get_exe_dir`, console/ANSI handling) without copying blind: learn the method, then write it.

Learning

A significant amount of use was **Socratic** — I asked the assistant to teach, not to answer. It quizzed me on pointer ownership, `struct` layout and padding, the memory model behind `malloc` / `free`, and how the Critical Path Method works. The exchange went **both ways**: the assistant quizzed me, and I **pushed back on its suggestions** — questioning, verifying, and rejecting the ones that did not hold up. The rule was symmetric — **each side had to defend its suggestion**, and the call was made on the **merit of the idea, not on who proposed it**. That argument is the learning, and it directly prepares me for the in-class oral exam, where I must explain every line.

Documentation & presentation

The assistant helped produce the function doc-comments, the architecture and data-model **diagrams** (as Graphviz `.dot` + Mermaid), this presentation, and this AI-usage document. The technical content is the project's own; the assistant helped structure and phrase it.

3. What was *me* vs. what was AI

- **Direction and decisions were mine.** I chose the idea, the architecture, the data structures, and the priorities. The assistant supplied options, drafts, and explanations.
- **I verified everything.** I read the generated code, required it to build clean, and ran the tests. I **caught and corrected real mistakes the AI made** — including one case where it overwrote my own edits, which I rejected and had restored.
- **I can explain every line.** Where I could not, I asked until I could — that was the point of using it Socratically.

The honest summary: AI was a **force multiplier** that let a single student build a much larger and more correct system than would otherwise fit the timeframe — but the engineering judgment, the verification, and the understanding are mine.

4. The policy question — leverage without losing ownership or the language

Using AI on a *learning* project raises a real tension: the same tool that shortcuts hours of manual labour can also shortcut the **understanding** the course is meant to build. If I let it write the pointer arithmetic, the `malloc` / `free` pairing, or the parser loop *for* me without engaging, I would ship a working program and learn nothing — and fail the oral exam, which is exactly designed to catch that.

I resolved it with a few explicit rules:

- **AI does labour, not judgment.** I let it generate the *tedious and repetitive* (a fifth near-identical menu loop, a field-by-field serializer, an HTML table) — but the *decisions* (data structures, ownership, algorithm choice, what gets split into which module) are mine. Mechanical $\neq$ conceptual; I only outsource the mechanical.
- **The "explain it back" gate.** I do not keep a piece of generated code I cannot explain line by line. When something was unclear (pointer decay, `const`-correctness in the finders, why the overlap resolver needs a fixpoint), I switched the assistant into **Socratic mode** and had it *quiz* me instead of answer — so the labour-saving tool also became the teacher.
- **Learn the hard parts by hand, automate the boring parts.** The core C ideas — the pointer model, manual memory management, the dependency graph, the CPM passes — I worked through myself, often with the assistant asking questions. The boilerplate around them I let it write. The skill being graded lives in the former; the hours saved are in the latter.
- **Verification is non-negotiable and mine.** Generated code is a *draft* until I have read it, built it at `/W4` with zero warnings, and run a test that proves it. AI-written code that I have not verified does not count as done.

The general principle: **AI multiplies output, but ownership and understanding have to be actively defended** — by drawing the labour/judgment line, by refusing to keep code I can't explain, and by doing the conceptually hard parts myself. Used that way, the tool accelerated the project *and* the learning instead of trading one for the other.